

115. Une place libre : tirer encore

Le hasard ne sait pas où sont les murs. On tire, on regarde, et on recommence tant que c'est pris.

1. Ce qu'il faut comprendre

On veut semer quinze pièces P dans une salle. Tirer une colonne et une ligne au hasard ne suffit pas : le hasard ne sait rien de l'écran. Il peut tomber **sur un mur**, ou **sur une pièce déjà posée** — on en verrait alors quatorze, et le jeu ne se finirait jamais.

Ce qui est nouveau ici : tirer jusqu'à trouver une case vide. C'est exactement le travail de `do ... while` (chapitre 3) : **au moins un** tirage, puis on recommence **tant que** la case n'est pas vide. Une case vide, c'est la tuile 0 : `lire(c, 1) != 0` veut dire « déjà prise ».

Déroulons une pièce. Premier tirage : (0, 5). `lire(0, 5)` rend un X, le bord : ce n'est pas 0, on retire. Deuxième tirage : (7, 9) : vide. On sort du `do ... while` et on pose le P. La pièce suivante ne pourra plus tomber en (7, 9) : `lire` y trouve un P.

Pourquoi ça finit ? Parce qu'il reste beaucoup de cases vides : sur $20 \times 18 = 360$ cases, il y a une soixantaine de murs et quinze pièces. Même au pire, un tirage sur deux ou trois tombe bien. **Si la salle était pleine, la boucle ne finirait jamais** : un jeu doit toujours garder de la place pour ce qu'il sème.

Tout se fait **écran éteint** (`ecran(0)`) : les quinze pièces et les murs apparaissent d'un coup, sans attendre une pause à chaque écriture.

À toi : remplace `do ... while` par un seul tirage, et compte les P : de temps en temps, il en manque.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ecran>` → [27.1. La fonction ecran\(\) — éteindre et rallumer l'écran](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <hasard>` → [0.8.1.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **523 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	58 o	← la console, à l'allumage — la mise en route de la console
main()	254 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case

```

AdresseCase    10 o ← texte(), lire() – trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
Reste          10 o ← main() – le reste d'une division
CopierTuiles   40 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Hasard         25 o ← hasard() – tire un nombre au hasard
Tuiles         32 o ← CopierTuiles – 2 lettres sur 43
"X"            1 o ← texte() – un texte gravé
"X"            1 o ← texte() – un texte gravé
"X"            1 o ← texte() – un texte gravé
"X"            1 o ← texte() – un texte gravé
"X"            1 o ← texte() – un texte gravé
"X"            1 o ← texte() – un texte gravé
"P"            1 o ← texte() – un texte gravé

```

Lettres gravées : PX

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte() × 6    105 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
ecran() × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
lire() × 1     11 o sur place, dans main() + AdresseCase (10 o)
hasard() × 2    6 o sur place, dans main() + Hasard (25 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : une salle bordée de X, avec un mur au milieu, et 15 pièces P
// semées au hasard – jamais sur un X, jamais deux sur la même case.
//
// Ce qui est nouveau : tirer ENCORE tant que la case est prise.
// do ... while : on tire au moins une fois, puis on recommence
// tant que lire(c, l) != 0 (0 = case vide ; autre chose = déjà prise).

#include <ecran>    // éteint ou rallume l'écran
#include <texte>    // écrit un texte à l'écran
#include <hasard>   // tire un nombre au hasard
#include <lire>     // lit la tuile posée sur une case
#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UNE CONSTANTE — PIECES

```

const uint8_t PIECES = 15;           // Le nombre de pièces à semer.

```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Tout se dessine écran éteint.

    // Le cadre : les bords de l'écran en X.
    for (uint8_t c = 0; c < 20; c++) { // chaque colonne :
        texte(c, 0, "X");              // en haut (ligne 0)
        texte(c, 17, "X");             // en bas (ligne 17)
    }
    for (uint8_t l = 1; l < 17; l++) { // chaque ligne de 1 à 16 :
        texte(0, l, "X");              // à gauche (colonne 0)
        texte(19, l, "X");             // à droite (colonne 19)
    }
    // Et un mur au milieu : ligne 8, colonnes 4 à 15.
    for (uint8_t c = 4; c < 16; c++) texte(c, 8, "X");

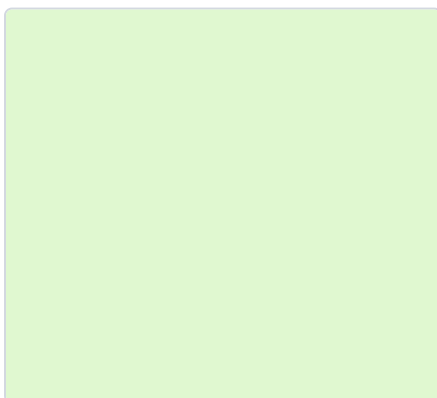
    // Quinze pièces, jamais sur un X, jamais sur un P.
    for (uint8_t i = 0; i < PIECES; i++) { // Pour chaque pièce :
        uint8_t c = 0;                    // sa colonne,
        uint8_t l = 0;                    // sa ligne.
        do {                               // FAIS :
            c = hasard() % 20;            // 0 à 19 : n'importe quelle colonne
            l = hasard() % 18;            // 0 à 17 : n'importe quelle ligne
        } while (lire(c, l) != 0);        // déjà prise ? on tire encore
        // Ex. : (0, 5) → un X → on retire ; (7, 9) → vide → on sort.
        texte(c, l, "P");                // ici, c'est vide : on pose
    }

    ecran(1);                             // On rallume : tout apparaît d'un coup.

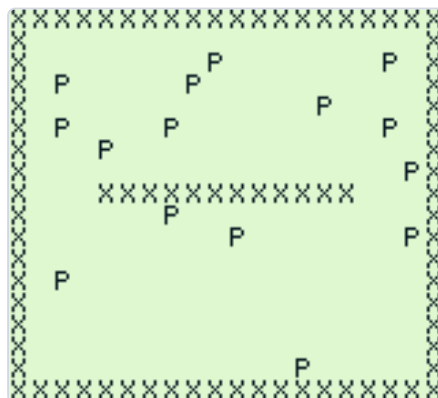
    while (true) {                         // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une salle bordée de X, un mur au milieu, et quinze P semés au hasard — jamais sur un X.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ exactement quinze P (15)

✓ aucun X recouvert : $20 + 20 + 16 + 16 + 12 = 84$ (84)

✓ le mur du milieu est entier

Ensuite : **116. Compter les secondes : un chronomètre** — chapitre 19 : Le hasard et le temps. · gameboy3 — écrire une cartouche Game Boy en C++